Technical Design Document / Low-Level Design (TDD/LLD)

Etel Ride Platform

Version 1.1 | Etel Group

Prepared for the INSA cyber-security audit application referenced in Etel Group's permission request letter to INSA's Director General, submitted in support of a Road Transport license application with the Ministry of Transport and Logistics. Companion to the SRS and SDD documents.

> Version 1.1 adds Section 6 (technology stack, key components, system architecture diagram, data flow diagrams and data flow explanation) requested by the INSA reviewer, supplied as PDF. It also corrects Section 3.1: the ride state machine now records `arrived` as a real status, verified against RideController.php. The five v1.0 documents are merged into Sections 1 to 5.

Scope note

Etel Ride shares one backend and one database with Etel Delivery (Etel Group's parcel/food delivery service line); there is no separate deployment. This document describes only the Ride-facing schema, API surface, and modules. Where a shared component (authentication, the admin capability system, hosting) also serves Delivery, that is stated explicitly rather than hidden; Delivery-only components (orders, items, service givers, couriers) are out of scope. All content is derived directly from the production source code at /delivery/ in the Etel-FTBG repository; every claim is traceable to a specific file, table, or action name.

Contents

1. Database Schema
2. API Contracts
3. State Machines
4. Module Breakdown
5. Security Controls
6. System Architecture, Technology Stack and Data Flow

1. Database Schema (Ride-relevant)

Etel Ride and Etel Delivery share one MySQL database and one schema-migration history (`api/migrations/001-057_*.sql`, plus `api/seeds/complete_migration.sql` for fresh installs). Only Ride-relevant tables are described below. Delivery-only tables (`orders`, `order_items`, `items`, `service_givers`, `delivery_zones`, `couriers`, etc.) are out of scope for this document.

1.1 Shared core tables (used by Ride)

Table	Key columns	Purpose
`users`	id, name, phone (unique), email, password_hash, `role`, lang, active, timestamps	Shared identity table. `role` distinguishes passenger (`customer`), `driver`, `admin`. Also used by Delivery for `courier` / `service_giver` roles — out of scope here.
`role_permissions`	role_name, capabilities (TEXT)	Admin capability system, shared with Delivery

1.2 Ride-specific tables added by numbered migrations

Migration	Table/change	Purpose
001	`login_attempts`	Brute-force/rate-limit tracking (shared with Delivery)
006	`users.token_version`	Bearer-token revocation counter (shared)
007	`ride_fare_rules`	Per-vehicle-type fare config (base_fare, per_km_rate, wait_rate_per_min, min_fare)
009	`driver_announcements`, `driver_announcement_reads`	Admin-authored driver news + read tracking
010	`ride_issues`	Passenger/driver-reported ride problems
017	`sos_alerts`	Emergency/SOS button backend (shared field for ride_id, reporter role)
024	`ride_favorite_routes`	Saved ride routes + referral `paid_at`
025	`rides.vehicle_type`	Ride table extension
026	`rides.surge_applied`	Ride table extension
028	`geocode_cache`	Cached lat/lng lookups
029	`rides.fare_breakdown` JSON	Ride table extension
030	`ride_path_points`	GPS breadcrumb trail per ride
031	`driver_vehicle_change_requests`	Admin-approved vehicle change workflow
032	`rides.last_location_at`	Ride table extension
033	Ride ratings/sharing/scheduling columns	Share links, scheduled rides
034	Driver commission tracking columns	Added to `ride_drivers`
037	`driver_shifts`	Shift tracking
038	`driver_fuel_logs`	Driver fuel-cost tracking

Migration	Table/change	Purpose
040–042	`equb_groups`, `equb_members`, `equb_contributions`, `equb_payouts`, `equb_participants`, `equb_packages`	Equb rotating-savings subsystem (shared with couriers, out of scope)
046	Hex-grid geospatial matching columns	Ride dispatch matching (`api/lib/hexgrid.php`)

1.3 Tables created inline by RideController.php (not in migrations/)

Table	Key columns	Purpose
`rides`	passenger_id, driver_id, pickup/dropoff lat/lng+address, distance_km, fare_estimate/final/paid, payment_method, `status` (created as ENUM; widened at runtime to VARCHAR(20) by `rides_add_missing_cols()` so that new states such as `arrived` fit) with values `searching`, `accepted`, `in_progress`, `arrived`, `completed`, `cancelled`, rating/rating_comment, expires_at/assigned_at/started_at/completed_at/cancelled_at	Core ride entity
`ride_drivers`	user_id (PK), online, lat/lng, vehicle_plate/type/make/color/year, license_number/name/dob, `identity_match ENUM('unverified','matched','mismatch')`, selfie_url, license_photo_url, `onboarded` flag, priority_score, total_earnings/paid_out/rides, rating_sum/count, last_seen, suspension columns	Driver profile & onboarding gate
`ride_drivers` (addition)		`etel_money_balance`: the driver's prepaid platform balance, from which the 5% commission is deducted on every completed ride (see `rides_complete_and_pay()` in `lib/ride_fare.php`)
`driver_payouts`	user_id, amount, partner_ref (unique), tx_ref, status	Driver cash-out records
`driver_etel_money_topups`	user_id, amount, tx_ref (unique), source, status, note, receipt_image_url, approved_by/at	Two-step-gated Etel Money top-up
`driver_compliance_requests`	driver_user_id, `doc_type ENUM('insurance','technical_inspection','road_fund')`, requested_expiry_date, requested_receipt_url, status, reject_reason, reviewed_at/by	Compliance document workflow
`driver_goal_tiers`	id, trips_required, bonus_etb, active, sort_order	Driver incentive tiers
`ride_messages` / `ride_typing`	ride_id, sender, message/typing state	In-ride chat

Also created inline, shared with Delivery (out of scope here): `app_release_channels` / `app_canary_testers` (staged rollout, used by every app including Ride/Driver), `order_issues` (Delivery only).

1.4 Notable design distinction

There is **no standalone `drivers` table shared with Delivery's courier concept**. `ride_drivers` (this document's scope) and `couriers` (Delivery, out of scope) are two entirely separate tables/roles, even though the same `users.id` could in principle hold either.

2. API Contracts (Ride-relevant)

Dispatch model

Single entrypoint: `api/boot.php`, shared with Etel Delivery. Every request carries an `action` field, dispatched to one of 17 controller files. Only Ride-relevant controllers/actions are detailed below.

Ride-relevant controller inventory

Controller	Actions in scope	Domain
`AuthController.php`	register, login, etelid_login, admin_etelid_login, change_password (5 of 6 — sg_login is Delivery-only)	Identity & session (shared)
`RideController.php`	65 actions	Ride-hailing lifecycle (fully in scope)
`AdminController.php`	Ride fare/pricing config, ride/driver issue resolution, driver account management (subset of 73 total actions)	Platform administration
`EqubController.php`	19 actions	Rotating savings (shared with couriers)
`AgentController.php`	agent_ride_quote, agent_book_ride, agent_ride_status, agent_cancel_ride (4 of 9 — the rest are Delivery-only)	EtelAI machine-to-machine booking
`NfcController.php`	nfc_start_ride (1 of 5 — the rest are Delivery-only)	EtelID NFC-tap bridge
`DriverFuelController.php`	4 actions	Driver fuel logs
`PushController.php`	3 actions	Web Push subscriptions (shared)

Out of scope (Delivery-only, not detailed here): `OrderController.php`, `ServiceGiverController.php`, `CourierFuelController.php`, and the Delivery-facing actions within `AdminController.php`, `AgentController.php`, and `NfcController.php`.

Authentication schemes in use (shared with Delivery)

1. **Bearer token** (most user-facing actions) — `Authorization: Bearer <token>`. Token = base64(`user_id:issued_at:token_version:hmac`), HMAC-SHA256 signed with `TOKEN_SECRET_FINAL`. 7-day hard expiry. `get_auth_user()` additionally compares the embedded `token_version` against the live `users.token_version` column for immediate revocation.
2. **Shared service key** (`AgentController`, `NfcController`) — `hash_equals()` constant-time check against `ETELID_SERVICE_KEY`, sent as a header with a `?key=` query-param fallback.

Worked request/response examples

`login`

```
POST /api/boot.php
{ "action": "login", "phone": "0912345678", "password": "••••••••" }

→ 200 OK
{ "success": true, "token": "<bearer token>", "user": { "id": 123, "name": "...", "role": "customer", ... }
}
```

`book_ride`

```
POST /api/boot.php   Authorization: Bearer <token>
{
  "action": "book_ride",
  "vehicle_type": "car",
  "pickup_lat": 7.06, "pickup_lng": 38.47, "pickup_address": "...",
  "dropoff_lat": 7.08, "dropoff_lng": 38.49, "dropoff_address": "...",
  "payment_method": "cash",
  "quote_token": "<from ride_estimate_fare>"
}

→ 200 OK
{ "success": true, "ride": { "id": 3301, "status": "searching", "fare_estimate": 85.00, "expires_at": "..."
} }
```

`driver_topup_etel_money`

```
POST /api/boot.php   Authorization: Bearer <token>   (role=driver, onboarded=1 required)
{ "action": "driver_topup_etel_money", "amount": 500.00, "tx_ref": "EP-...", "receipt_image_url": "..." }

→ 200 OK
{ "success": true, "topup": { "id": 771, "status": "pending", "amount": 500.00 } }
```

Note: the balance is **not** credited on this response — `tx_ref` is server-verified via `etel_pay_verify_tx()` before the row is even created as `pending`, and a separate admin action (`admin_approve_etel_topup`) must approve it before `ride_drivers.total_earnings`/wallet balance actually moves.

Real-time / streaming interfaces

Endpoint	Transport	Purpose
`sse_ride_status`	Server-Sent Events	Live ride status push to passenger/driver
`sse_driver_stream`	Server-Sent Events	Live driver location push

External interface contracts (Ride-relevant)

System	Actions/files	Auth
EtelID SSO	`etelid_login`, `admin_etelid_login`	User-driven redirect flow
EtelID NFC bridge (ride)	`NfcController.php::nfc_start_ride`	Shared service key
EtelPay	`etel_pay_verify_tx()` call site in `RideController.php`	Server-to-server verification call
Telegram	`api/telegram_webhook.php`	Bot token

3. State Machines (Ride-relevant)

3.1 Ride Lifecycle (`rides.status`)

Values in use: `searching` , `accepted` , `in_progress` , `arrived` , `completed` , `cancelled` . The column was created as an ENUM of five values; `rides_add_missing_cols()` widens it to `VARCHAR(20)` at runtime so that `arrived` (and any future state) fits. Version 1.0 of this document described `arrived` as a sub-state of `accepted` ; reading `driver_arrived` , `driver_complete_ride` and `customer_confirm_arrival` in `RideController.php` shows that it is a real status, reached at the end of the trip.

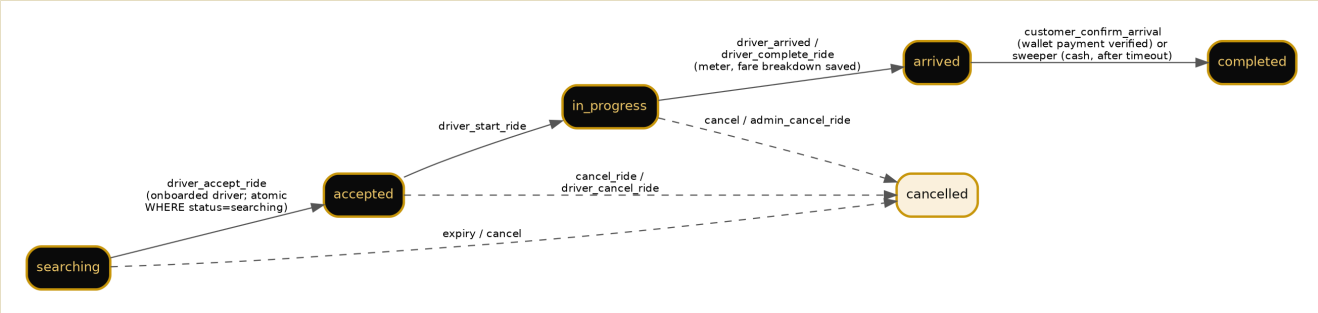


Figure 4 - Ride lifecycle (rides.status)

Transition rules (each is a single guarded `UPDATE ... WHERE status = <expected>` , so a duplicate or replayed action matches no row and is rejected):

From	To	Action	Guard
(new)	`searching`	`book_ride`	authenticated passenger; fare computed server-side
`searching`	`accepted`	`driver_accept_ride`	`rides_require_onboarded()` ; driver has no active ride; driver not excluded for this ride; `WHERE status='searching'` makes acceptance atomic, so only one driver wins (otherwise 409)
`accepted`	`in_progress`	`driver_start_ride`	`WHERE id=? AND driver_id=? AND status='accepted'`
`in_progress`	`arrived`	`driver_arrived` or `driver_complete_ride`	`WHERE driver_id=? AND status='in_progress'` ; any running wait clock is stopped, distance, fare and `fare_breakdown_json` are stored, `arrived_at` is set. `driver_complete_ride` now behaves like `driver_arrived` so no ride is completed without the passenger's confirmation
`arrived`	`completed`	`customer_confirm_arrival`	`WHERE passenger_id=? AND status='arrived'` ; for wallet payment `etel_pay_verify_tx()` must verify the reference and a reference cannot be reused for another ride (409). Completion goes through `rides_complete_and_pay()`
`arrived`	`completed`	stuck-trip sweeper (`lib/ride_sweeper.php`)	cash rides only, after `ride_auto_complete_minutes` (default 30); wallet-paid rides instead get an open `ride_issues` row for an admin
`in_progress`	`arrived`	stuck-trip sweeper	no movement for `ride_idle_end_minutes` (default 180); flagged with an open `auto_ended` issue

From	To	Action	Guard
`searching` / `accepted` / `in_progress`	`cancelled`	`cancel_ride`, `driver_cancel_ride`, `admin_cancel_ride`, expiry	caller must own the ride, or hold the admin capability

Payment on completion. `rides_complete_and_pay()` (`api/lib/ride_fare.php`) is the single completion routine, shared by `customer_confirm_arrival`, the sweeper and `admin_manual_complete_ride`, so payout maths cannot drift between paths. Only wallet-paid rides credit `ride_drivers.total_earnings` (cash was already collected in person). On every completed ride, cash or wallet, a 5% commission is deducted from the driver's prepaid `etel_money_balance`.

Waiting fee. The wait clock (`wait_active`, `wait_started_at`, `wait_seconds_total`) is started and stopped by `driver_toggle_wait` while the ride is `in_progress`; it is stopped automatically when the driver ends the trip, and billable waiting time is capped by `ride_wait_cap_minutes` (default 30).

3.2 Driver Onboarding Gate

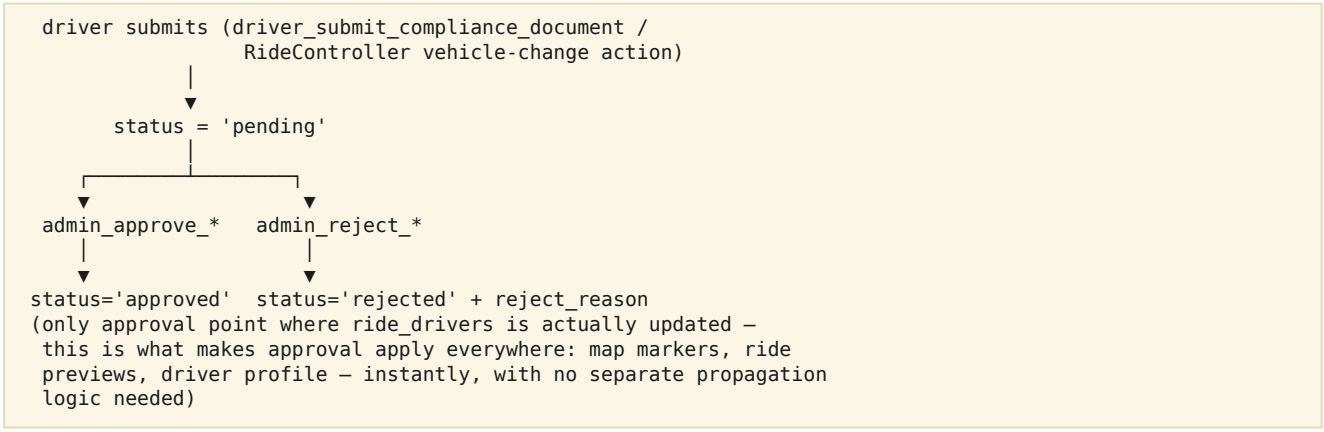
```
register / etelid_login  →  users.role = 'customer' (always, regardless of intent)
                                        │
                            driver_update_profile (submit vehicle info,
                            selfie, license — NOT gated, deliberately open
                            so onboarding can begin)
                                        │
                                        ▼
                          ride_drivers row created, onboarded = 0
                                        │
                    EtelID identity cross-check + admin/system review
                                        │
                                        ▼
                          ride_drivers.onboarded = 1
                          users.role updated to 'driver'
                                        │
                                        ▼
       rides_require_onboarded() now passes — the following unlock:
       driver_go_online, driver_get_nearby_requests, driver_accept_ride,
       driver_hail_customer, driver_start_ride, driver_arrived,
       driver_complete_ride, driver_cashout, driver_topup_etel_money
```

This single shared gate function (`rides_require_onboarded()`) was introduced to close a fraud/safety gap: previously, any authenticated role — including a plain, non-onboarded `customer` — could call these driver-privileged actions with no onboarding check at all.

3.3 Etel Money Top-up (two-step approval)

```
driver_topup_etel_money (driver submits tx_ref + receipt)
            │
            ▼
etel_pay_verify_tx()  ──fails──►  rejected, no row created
            │ succeeds
            ▼
driver_etel_money_topups row created, status = 'pending'
            │
            ▼
admin_approve_etel_topup (admin, separate step)
            │
            ▼
status = 'approved'  →  balance credited to ride_drivers.total_earnings/wallet
```

3.4 Compliance Document / Vehicle Change Approval (shared pattern)



4. Module Breakdown (Ride-relevant)

Backend (`delivery/api/`) — Ride-relevant files

Path	Role
`boot.php`	Single dispatch entrypoint, auth, rate limiting — shared with Delivery
`controllers/AuthController.php`	Identity & session — shared with Delivery
`controllers/RideController.php`	Ride-hailing lifecycle (65 actions, ~210KB, largest module in the codebase)
`controllers/AdminController.php`	Platform administration — only its ride fare/pricing and ride/driver-issue-resolution actions are in scope; its order/zone/service-giver actions are Delivery-only
`controllers/EqubController.php`	Rotating savings (19 actions, shared with couriers)
`controllers/AgentController.php`	EtelAI machine-to-machine booking — only its ride_* actions are in scope
`controllers/NfcController.php`	EtelID NFC-tap bridge — only nfc_start_ride is in scope
`controllers/DriverFuelController.php`	Fuel logs (4 actions)
`controllers/PushController.php`	Web Push subscriptions (3 actions, shared)
`lib/ride_fare.php`	Ride fare calculation (shared by RideController + AgentController)
`lib/hexgrid.php`	Geospatial hex-grid driver matching
`lib/schema_helpers.php`	Idempotent runtime schema healing (`rides_add_missing_cols()`)
`lib/app_releases.php`	Staged release-channel/canary management, shared across all apps
`etelid_auth.php`	EtelID SSO cookie-session bridge (shared)
`telegram_webhook.php`	Telegram bot integration for operational alerts (shared)
`updater.php` / `updater-api.php`	Production file-deploy tool (shared)
`migrate.php`	Migration runner (shared)

Out of scope (Delivery-only): `controllers/OrderController.php`, `controllers/ServiceGiverController.php`, `controllers/CourierFuelController.php`, `lib/delivery_pricing.php`.

Frontend (`delivery/`) — Ride-relevant apps

App	Entry point(s)	Role served
Ride (passenger)	`ride/index.html`, `ride/share.html`	Passenger
Driver	`driver/index.html`	Driver
Equb	`equb/index.html`	Driver (benefit feature, shared with couriers)
Offline fallback	`offline.html`	All (service worker navigation fallback)

Each installable app has its own manifest, is independently versioned, and is packaged as an Android Trusted Web Activity (TWA).

Out of scope (Delivery-only): `customer.html` / `index.html`, `courier/`, `servicegiver.html`, `track.html`. Note: `admin.html` and `login.html` are shared entry points used by every role including Ride/Driver — they are not themselves Delivery- or Ride-exclusive.

Ride-relevant admin front-end areas (within the shared `admin.html`)

Nav page	Backing controller
Rides	RideController (admin actions)
Pricing	AdminController (`ride_fare_rules` subset)
Support	AdminController (ride/driver issue resolution, SOS)
Releases	`lib/app_releases.php` (Ride/Driver channels)
Equb	EqubController

`admin.html` also has Delivery-only nav pages (Orders, Couriers, Service Givers, delivery Pricing) and platform-wide pages unrelated to either service line (Roles, Finance & Planning, Danger Zone, Documents) — out of scope here.

Build pipeline (shared)

`js/build-bundles.js` (terser) compiles per-app source files into minified `-bundle.js` files, synced into versioned `js/releases/{app}/{version}/` folders managed by the release-channel system. Source edits alone do not change production behavior until this rebuild-and-promote step completes.

5. Security Controls (Implementation Level)

This section documents concrete implementation-level security controls as they apply to Etel Ride. Most of these mechanisms (authentication, injection prevention, rate limiting) are shared infrastructure also used by Etel Delivery — they are described here because Ride depends on them, not because they are Ride-exclusive.

5.1 Authentication (shared)

- Custom signed bearer-token scheme (`api/boot.php`), not PHP sessions, not JWT.
- `gen_token($user_id, $version)`: packs `user_id:issued_at:token_version`, HMAC-SHA256-signs with `TOKEN_SECRET_FINAL`, base64-encodes.
- `verify_token()`: hard 7-day lifetime; malformed tokens rejected closed; signature compared with timing-safe `hash_equals()`.
- `get_auth_user()`: additionally compares the token's embedded `token_version` against the live `users.token_version` column — a mismatch invalidates the token immediately, independent of the 7-day window. This is the server-side revocation mechanism.
- No silent token refresh exists anywhere — expiry always produces a clean 401 requiring re-login.
- `logout` calls a helper that increments `token_version`, invalidating every outstanding token for that user across every device in one write.
- Exactly one cookie-based auth surface exists in the whole codebase: `api/etelid_auth.php` (bridging to EtelID's SSO), using `Secure` (conditional on real HTTPS detection) + `HttpOnly` + `SameSite=Lax` + an 8-hour inactivity timeout.
- Service-to-service auth (`AgentController`'s ride actions, `NfcController`'s `nfc_start_ride`) uses a separate shared-secret scheme (`ETELID_SERVICE_KEY`, `hash_equals()`), independent of the bearer-token system, with a `?key=` query-param fallback alongside the header.

5.2 Authorization (Ride-specific)

- Driver onboarding gate: `rides_require_onboarded()` — a single shared function gating every driver-privileged ride action on `ride_drivers.onboarded = 1`, closing a fraud/safety gap where any authenticated role could otherwise accept and run a real ride.
- Ownership checks: ride-scoped actions verify the caller's own passenger_id/driver_id matches the ride being acted on; shared/two-party actions (e.g. `send_ride_message`) compute the caller's role from whichever column matches and reject with 403 if neither does.
- `nfc_start_ride` independently re-verifies the resolved driver's onboarding status server-side, so the NFC path cannot be used to route around the same gate the phone-entry ride flow enforces.
- Admin capability model (shared): ride-relevant admin actions are gated through `require_admin($pdo, 'capability')`, checking both the `admin` role and a specific capability string (e.g. `pricing.manage`) against the `role_permissions` table.

5.3 Injection Prevention (shared)

- All database access uses PDO parameterized queries with `PDO::ATTR_EMULATE_PREPARES => false`.
- Any dynamic column/table name always traces to a hardcoded whitelist, never to request input; `LIMIT`/date-range values are always `intval()`'d or selected from fixed fragments.

5.4 Output Encoding (XSS Prevention) — Ride-facing surfaces

- All user-controllable values rendered via `innerHTML` or equivalent in `driver.js`/`ride.js` are passed through `escapeHtml()`/`esc()` helpers (shared helpers defined in `js/core.js`).

- Stored-XSS sinks in ride-history detail modals (`driver.js`, `ride.js`) were found and fixed during the codebase's internal security review.
- `escUrl()` helper rejects `javascript:` / `vbscript:` / `data:text/html` schemes on any raw image/photo URL placed into an attribute (applied to `driver.js` / `ride.js` among other app files).
- Fixes are propagated into the versioned release bundles (`js/releases/{app}/{version}/`), not just source.

5.5 File Upload Handling (Ride-relevant: driver documents)

- `upload.php`: content-sniffed MIME-type whitelist (not filename extension alone), size cap, whitelisted destination folders, fully server-generated filenames (client filename never used) — used for driver selfie, license photo, and compliance-document uploads.

5.6 Rate Limiting (shared)

- `login_attempts` table + `rate_limit_check()` / `rate_limit_fail()` / `rate_limit_clear()` helpers in `boot.php`, applied to `login`, keyed by `login:<phone>`.

5.7 Error Handling & Logging (shared, spot-checked on Ride-relevant files)

- `change_password`'s token-refetch failure hard-fails (`require_relogin: true`) rather than soft-degrading with a possibly stale token — applies to driver and passenger accounts alike.

5.8 Web-Server-Level Controls (shared)

- `Options -Indexes` set repo-wide; `api/backups/` (runtime deploy backups created by the Updater tool) has its own explicit `Require all denied` `.htaccess`.
- Updater tool credentials load from `.env` / `getenv()` and fail closed (HTTP 503) if unset — no default password, no silent bypass.

5.9 Payment Integrity (Ride-relevant: driver earnings)

- Driver Etel Money top-ups: `tx_ref` is server-verified via `etel_pay_verify_tx()` against EtelPay before the top-up row is even created as `pending`; balance only moves after a **second**, independent admin approval step (`admin_approve_etel_topup`) — a single compromised or spoofed `tx_ref` cannot move funds without both checks passing.

6. System Architecture, Technology Stack and Data Flow

This section was added in version 1.1 at the INSA reviewer's request. It gives the implementation-level technology stack, key components, system architecture and data flows that the schema, API contracts, state machines and security controls in Sections 1 to 5 sit within.

6.1 Technology Stack

Layer	Technology	Where it is used
Client	HTML5, CSS3, vanilla JavaScript (no framework); Progressive Web App; installable as Android Trusted Web Activity (TWA)	ride/, driver/, admin.html, login.html; per-app manifest files
Client build	Node.js + terser: per-app source compiled to minified bundles, synced to versioned release folders (stable/canary channels)	js/build-bundles.js, js/releases/{app}/{version}/
Offline / caching	Service worker: cache-first static assets, network-first API calls with 5-minute cache, offline fallback page	sw.js, offline.html
Maps and location	Google Maps JavaScript API (map, markers, polylines, Places Autocomplete), Geocoding API, browser Geolocation	ride and driver apps; server-side geocoding helpers
Server language	PHP 8.1 (procedural, controller-dispatch); no third-party framework	api/boot.php, api/controllers/*
Database	MySQL / MariaDB, utf8mb4, accessed through PDO with prepared statements (emulated prepares disabled); session time zone aligned with PHP (Africa/Addis_Ababa)	api/boot.php, api/migrations/*.sql
Authentication	Custom HMAC-SHA256 signed bearer token with token_version revocation; EtelID SSO; shared service key (hash_equals) for machine-to-machine calls	boot.php, AuthController, etelid_auth.php
Real-time	Server-Sent Events (SSE) for live ride status and driver location	sse_ride_status, sse_driver_stream
Notifications and alerting	Web Push with VAPID (RFC 8292); Telegram Bot API for operational alerts	api/webpush.php, push.php, telegram_webhook.php
Payments integration	EtelPay server-to-server transaction verification (etel_pay_verify_tx)	api/etel_pay_bridge.php
Background jobs	cPanel cron calling key-protected PHP scripts	cron_close_stuck_rides.php, cron_dispatch_scheduled.php, equb_cron.php
Hosting and deployment	Shared cPanel hosting (Apache/LiteSpeed), HTTPS; no SSH; deploys through in-app Admin Updater with backup and patch_log; GitHub Actions workflow for zip-on-merge	api/updater.php, .github/workflows/deploy-zip-on-merge.yml

6.2 Key Components

Component	Type	Responsibility	Trust boundary / security role
Ride app (passenger)	Client PWA/TWA	Registration and login, fare estimate, booking, live tracking, chat, SOS, ratings, favourite and scheduled rides	Untrusted. Holds bearer token only; every action re-authorised server-side.
Driver app	Client PWA/TWA	Onboarding, go online/offline, GPS pings, accept/arrive/start/complete, earnings, cash-out, top-up	Untrusted. Privileged actions gated by rides_require_onboarded().
Admin console	Client web app	Pricing, driver approvals, vehicle-change review, issue and SOS handling, release channels	Untrusted client; each action requires admin role plus capability via role_permissions.

Component	Type	Responsibility	Trust boundary / security role
boot.php	Gateway	Request parsing, token / service-key authentication, rate limiting, action whitelist, dispatch	Single choke point for all API traffic.
Controllers	Business logic	Auth, Ride, Admin, Equb, Agent, Nfc, DriverFuel, Push	Ownership checks per ride; capability checks per admin action.
ride_fare.php / hexgrid.php	Library	Fare calculation shared by the app and EtelAI; hex-grid nearest-driver matching	Pure computation; no external input trusted.
SSE endpoints	Real-time	Push ride status and driver position to the participants	Authenticated; scoped to the caller's own ride.
upload.php	Service	Driver selfie, licence and document uploads	Content-sniffed MIME whitelist, size cap, server-generated file names.
Cron scripts	Background	Close stuck rides, dispatch scheduled rides, Equb processing	Standalone, protected by a secret key from .env.
MySQL database	Data store	Users, rides, drivers, GPS path, fares, payouts, permissions	Reached only through PDO prepared statements.
EtelID, EtelPay, EtelAI	External group systems	Identity / SSO, wallet verification, AI booking agent	Shared service key over HTTPS, constant-time comparison.
Google Maps, Telegram, Web Push	External third parties	Maps and geocoding, operational alerts, browser notifications	API key / bot token / VAPID key held server- or domain-restricted.

6.2.1 Component inventory (files)

Component	Responsibility
api/boot.php	Shared dispatch entrypoint: request parsing, auth (token or service key), rate limiting, action routing. Serves both Ride and Delivery.
api/controllers/RideController.php	Ride-hailing lifecycle: fare estimation, booking, driver matching, ride state machine, driver earnings/compliance
api/controllers/AuthController.php	Identity & session (shared with Delivery)
api/controllers/AdminController.php	Platform administration; only its ride fare/pricing, driver-account, and ride/driver-issue-resolution actions are in scope here. Its order/service-giver/zone management actions serve Delivery only and are out of scope for this document.
api/controllers/EqubController.php	Rotating-savings benefit, used by drivers (and, out of scope here, couriers)
api/controllers/AgentController.php	EtelAI machine-to-machine integration; only its ride_* actions are in scope
api/controllers/NfcController.php	EtelID NFC-tap bridge; only nfc_start_ride is in scope (its courier-pickup/delivery actions serve Delivery)
api/lib/ride_fare.php	Ride fare calculation, shared identically between the Ride app and the EtelAI agent integration
api/lib/hexgrid.php	Geospatial hex-grid driver matching
Ride app / Driver app	The two Ride-facing PWAs, each independently versioned and installable as an Android TWA
sw.js (service worker)	Offline caching, install/activate lifecycle, cache versioning (shared mechanism, per-app cache namespace)

Component	Responsibility
EtelID (external)	Group-wide identity provider; SSO login, driver identity cross-check, NFC-tap bridge
EtelPay (external)	Wallet/payment app; verifies driver Etel Money top-up transactions

6.3 System Architecture Diagram

Etel Ride is a four-tier system. Thin PWA/TWA clients talk over HTTPS to a single PHP gateway (boot.php), which authenticates every request and dispatches it to a controller. Controllers read and write the MySQL database and call a small set of external services. Nothing in the client tier is trusted: authorization, ownership and fare calculation are all enforced on the server.

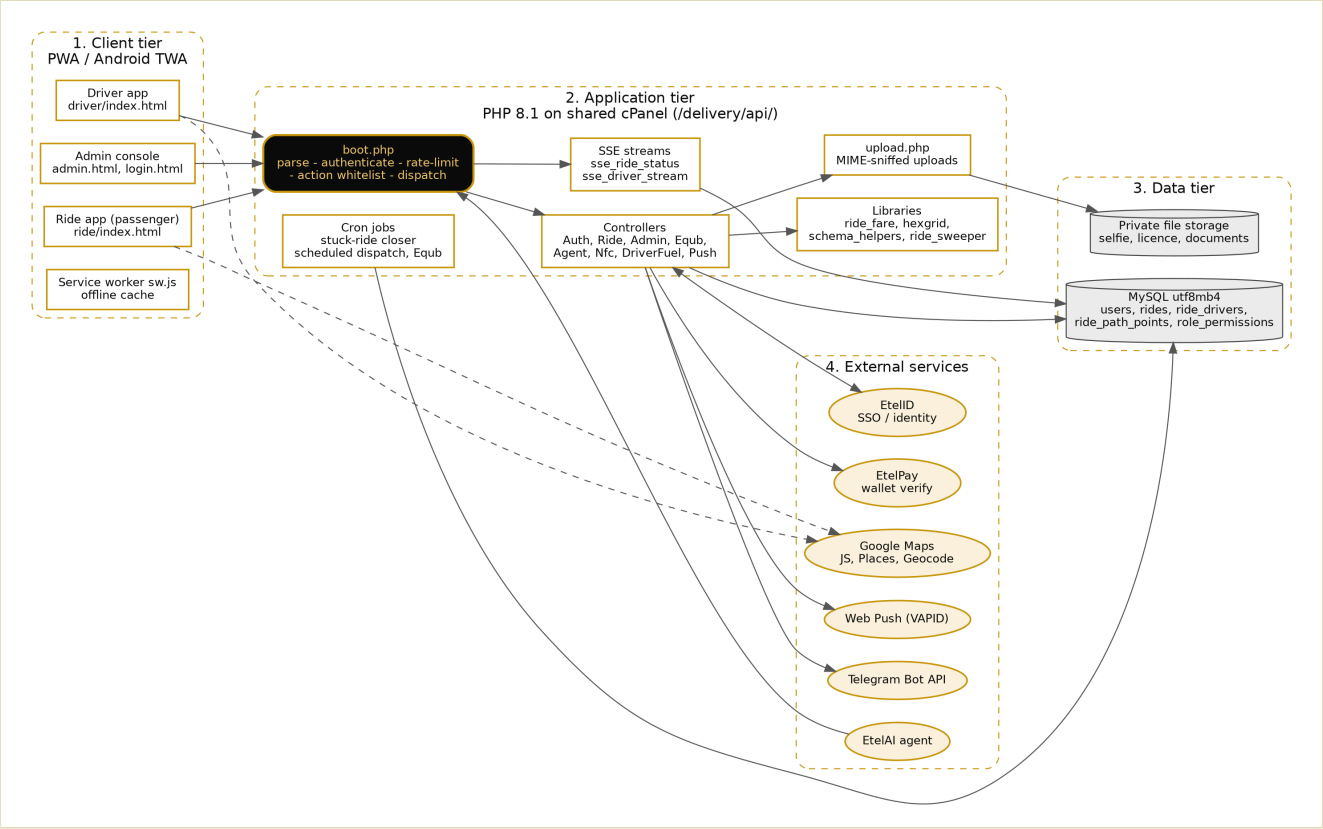


Figure 1 - Etel Ride system architecture (dashed arrows: browser-direct calls to Google Maps)

6.4 Data Flow Diagrams

6.4.1 Level 0 - context diagram

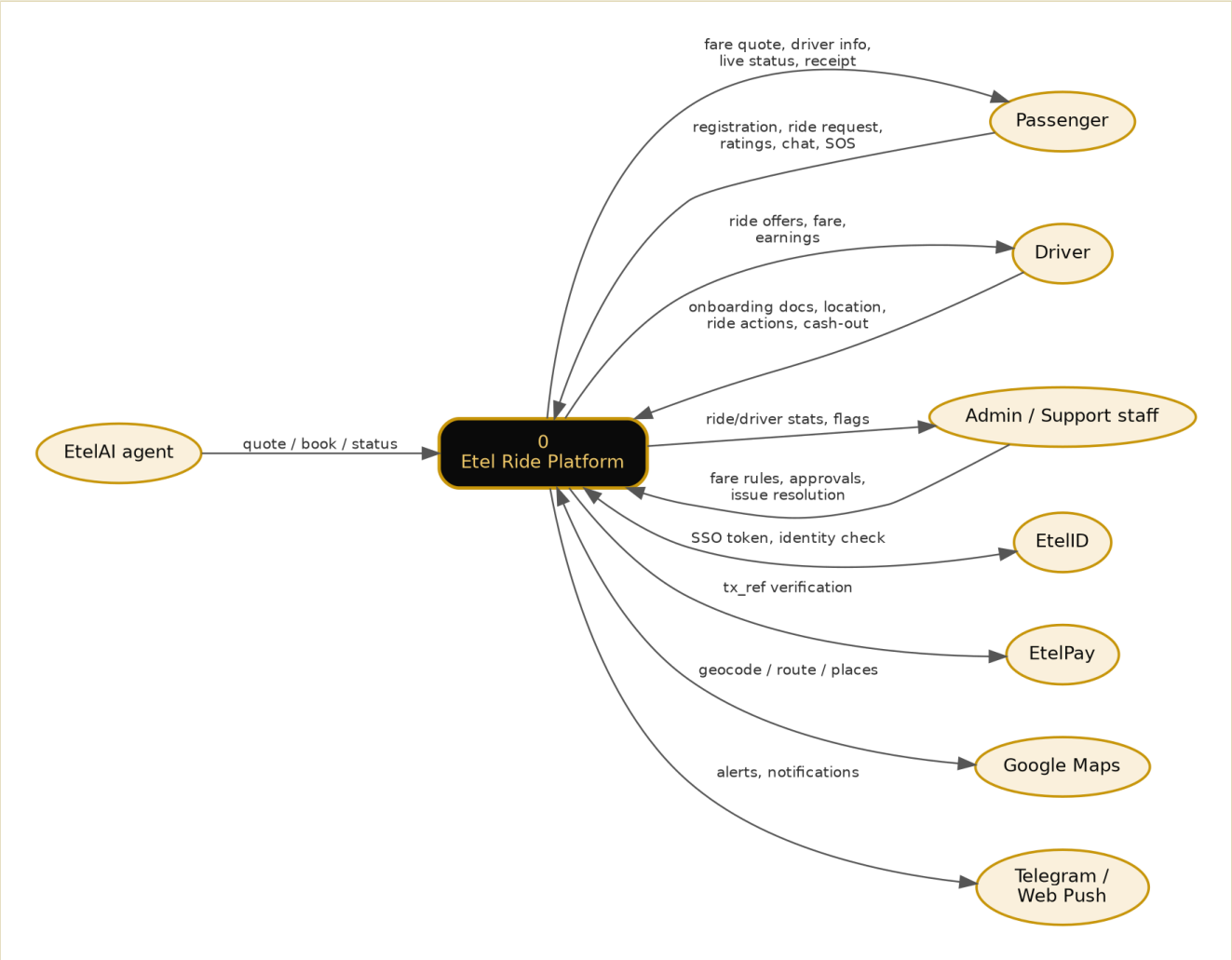


Figure 2 - Level 0 DFD: Etel Ride and its external entities

6.4.2 Level 1 - process decomposition

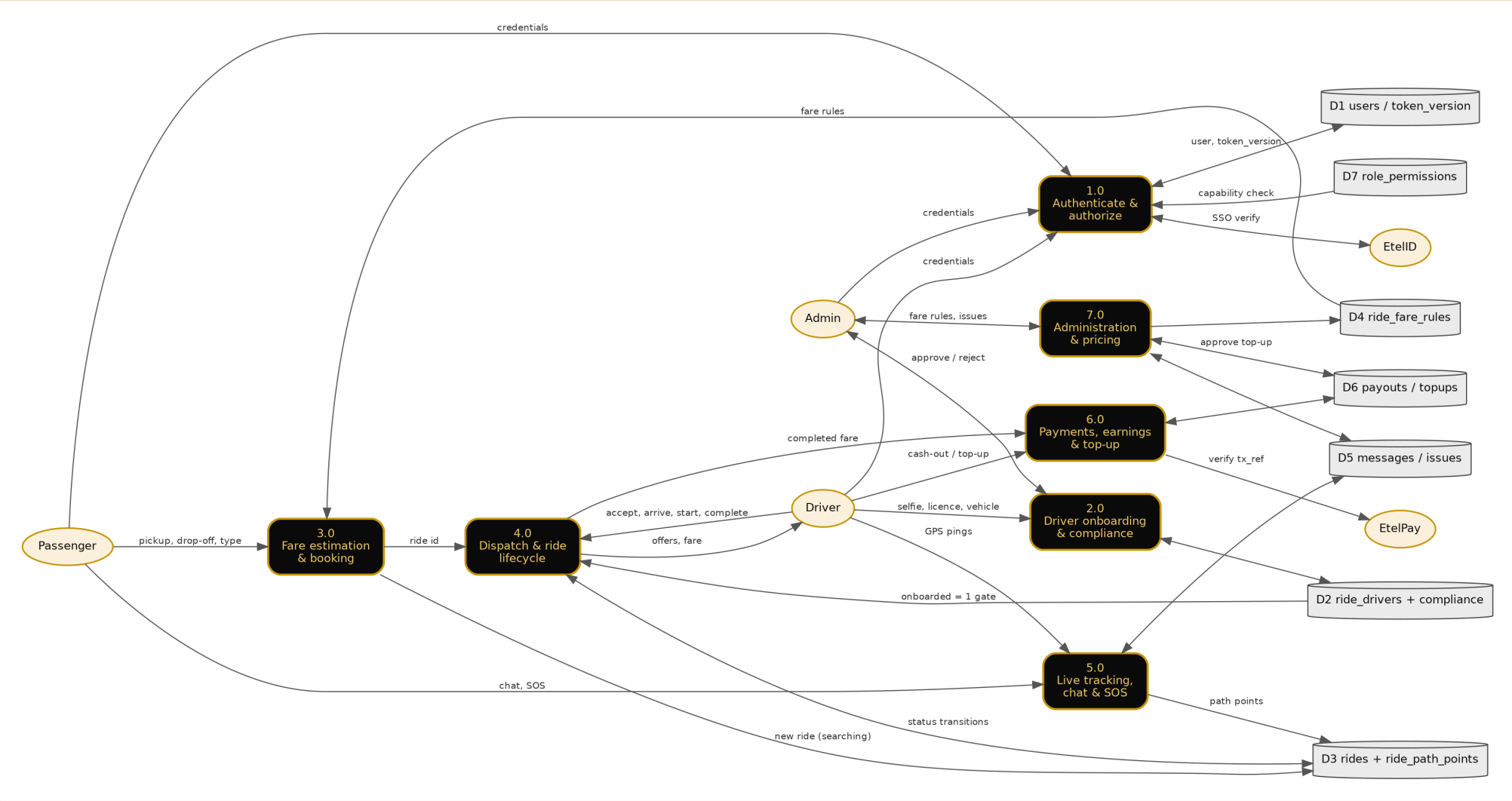


Figure 3 - Level 1 DFD: processes 1.0 to 7.0, data stores D1 to D7, external entities

6.5 Data Flow Explanation

6.5.1 External entities

Entity	Sends to Etel Ride	Receives from Etel Ride
Passenger	Registration, login, pickup and drop-off, vehicle type, chat, SOS, ratings	Fare quote, nearby drivers, driver details, live status and location, receipt
Driver	Onboarding documents (selfie, licence, vehicle), GPS pings, ride actions, cash-out and top-up requests	Ride offers, fare, earnings, announcements
Admin / support staff	Fare rules, driver and vehicle-change decisions, issue resolution, top-up approval	Ride and driver statistics, flagged identities, SOS and issue queues
EtelID	SSO token verification result, role check, NFC-tap ride start	SSO token, driver identity cross-check request
EtelPay	Transaction verification result	tx_ref, expected sender and amount
Google Maps	Map tiles, places, geocode results	Coordinates and address queries
EtelAI agent	Quote, book, status and cancel requests (service key)	Quote, ride status
Telegram / Web Push	-	Operational alerts and rider/driver notifications

6.5.2 Processes

1. **1.0 Authenticate and authorize.** The user submits phone and password (or an EtelID SSO token). Login is rate-limited (login_attempts, keyed by phone). On success the server issues an HMAC-SHA256 bearer token embedding user id, issue time and the user's current token_version. Every later request passes through boot.php, which verifies the signature (hash_equals), the 7-day lifetime and that token_version still matches D1, so logout or password change revokes all tokens at once. Admin actions additionally check D7 (role_permissions) for the required capability.
2. **2.0 Driver onboarding and compliance.** The driver uploads selfie, licence photo and vehicle details through upload.php (content-sniffed MIME whitelist, size cap, server-generated names). The identity is cross-checked with EtelID; mismatches are flagged for admin review. Admin approval sets ride_drivers.onboarded = 1 in D2. Later vehicle changes create a pending request and only take effect when an admin approves.
3. **3.0 Fare estimation and booking.** The passenger sends pickup, drop-off and vehicle type. ride_fare.php reads the per-vehicle rules from D4, computes the fare server-side (client values are never trusted) and returns a quote. On booking, a ride row is written to D3 with status searching.
4. **4.0 Dispatch and ride lifecycle.** hexgrid.php finds nearby online drivers. A driver must pass the onboarding gate (D2) before accepting. The ride moves searching, accepted, in_progress, arrived, completed or cancelled in D3; every transition checks that the caller owns the ride and that the previous status is valid, so a duplicate or replayed action is rejected. Stuck rides are closed by the sweeper and cron script.
5. **5.0 Live tracking, chat and SOS.** The driver app posts GPS points, stored as ride_path_points in D3. SSE endpoints stream status and position to the two ride participants only. Chat messages, issues and SOS events are stored in D5; SOS and operational alerts are forwarded to staff through Telegram and Web Push.
6. **6.0 Payments, earnings and top-up.** On passenger confirmation (or the sweeper timeout for cash rides) the fare is posted to the driver's earnings and the 5% commission is deducted from the driver's Etel Money balance. For an Etel Money top-up the driver supplies a tx_ref, which is verified server-to-server against EtelPay before a pending row is created in D6; the balance moves only after a second, independent admin approval. Cash-outs are recorded in D6.

7. **7.0 Administration and pricing.** Capability-scoped admin actions edit fare rules (D4), resolve issues (D5), approve drivers and vehicle changes (D2) and approve top-ups (D6). Release channels control which app bundle version users receive.

6.5.3 Typical end-to-end flow (ride request)

1. Passenger app sends ride_estimate_fare to boot.php with its bearer token. boot.php authenticates, whitelists the action and dispatches to RideController.
2. RideController reads D4, computes the quote and returns it. The passenger confirms and book_ride writes a searching ride to D3.
3. Nearby drivers receive the request. A driver calls driver_accept_ride; the onboarding gate and an atomic status update ensure only one driver wins.
4. driver_start_ride moves the ride to in_progress; driver_update_location records the path while SSE keeps the passenger updated.
5. At the drop-off the driver calls driver_arrived (or driver_complete_ride), which stores the fare breakdown and moves the ride to arrived. The passenger confirms with customer_confirm_arrival (wallet payments are verified with EtelPay first); rides_complete_and_pay() then completes the ride, credits wallet-paid earnings, deducts the 5% commission from the driver's Etel Money balance, and the passenger can rate. If the passenger never confirms, the sweeper completes cash rides after a timeout.

6.5.4 Data in transit and at rest

- All client-server and server-server traffic uses HTTPS; bearer tokens travel in the Authorization header only.
- Passwords are stored hashed; tokens are not stored server-side, only the per-user token_version.
- All SQL is parameterised; dynamic identifiers come from fixed whitelists.
- Uploaded documents are stored under server-generated names; runtime deploy backups are blocked from web access (Require all denied).
- Service-to-service calls use a shared key compared with hash_equals(), with a query-parameter fallback for hosts that strip custom headers.